

Machine Learning and Agentic AI for Macroeconomic Research

6: Heterogeneous Agent Models via Reinforcement Learning

Zhigang Feng

Spring 2026

Lecture Agenda

1. **Aiyagari–Bewley–Huggett** — incomplete-markets HA model and stationary equilibrium
2. **Computation of stationary equilibrium** — the classical three-step algorithm
3. **Why ML?** — motivating example on optimal growth, neural-network approximation
4. **Euler-equation methods** — residual losses, supervised learning, actor–critic
5. **Global solution via DNN** — distribution as state, value/policy networks
6. **Krusell–Smith (1998)** — aggregate uncertainty and bounded-rationality moments
7. **DeepHAM and DEQN** — modern deep-learning solutions for HA models
8. **OLG with aggregate uncertainty** — life-cycle structure, homotopy training
9. **Mean-Field Games** — continuous-time HA, HJB & KFE, link to RL
10. **Lab preview** — Krusell–Smith via Deep Equilibrium Nets in PyTorch

Aiyagari–Bewley–Huggett Setup

Why heterogeneous agents?

- The representative-agent paradigm assumes *complete markets* and a single decision maker.
- Many empirically important questions require **heterogeneity**:
 - Wealth and income inequality
 - Precautionary savings and the demand for insurance
 - Distributional consequences of monetary and fiscal policy
 - Heterogeneous responses to aggregate shocks
- HA models combine:
 - Idiosyncratic income (or productivity, mortality, ...) shocks
 - Incomplete asset markets (a single risk-free bond, borrowing constraint)
 - General equilibrium in factor prices
- This lecture: Aiyagari → Krusell–Smith → deep-learning solutions → Mean-Field Games.

Aiyagari (1994, QJE): heterogeneity among households

- A continuum of *ex-ante identical*, infinitely-lived households.
- Each receives an idiosyncratic, uninsurable labour-productivity shock ϵ_t following a Markov process.
- Households self-insure by accumulating a single risk-free asset a_t .
- A natural or ad-hoc **borrowing constraint** $a_t \geq -\underline{a}$ keeps the asset distribution well-defined.
- Despite ex-ante symmetry, the realized history of shocks generates an **ex-post** distribution of wealth.
- Bewley (1980s, unpublished), Huggett (1993), Aiyagari (1994): the workhorse incomplete-markets HA framework.

General equilibrium with ex-post heterogeneity

- **Idiosyncratic shocks:** $\epsilon_t \in \mathcal{E}$, transition $\pi(\epsilon, \epsilon')$, with $\sum_{\epsilon} \pi^*(\epsilon) = L$ constant.
- **No aggregate shocks** in Aiyagari (we add them with Krusell–Smith later).
- **Markets:**
 - Risk-free bond (or capital) at price $1/(1+r)$
 - Labour supplied inelastically; wage w
- **Stationarity:** prices (r, w) and the cross-sectional distribution $\lambda(a, \epsilon)$ are constant in time.
- **Individual state:** $(a, \epsilon) \in \mathbb{R}_+ \times \mathcal{E}$.
- **Aggregate state:** distribution λ over individual states (in stationary equilibrium, $\lambda = \lambda^*$).

Household's recursive problem

For given prices (r, w) and stationary distribution λ , each household solves

$$v(a, \epsilon; \lambda) = \max_{c, a'} \left\{ u(c) + \beta \sum_{\epsilon'} v(a', \epsilon'; \lambda) \pi(\epsilon, \epsilon') \right\}$$

subject to

$$c + a' = (1 + r) a + w \epsilon, \quad a' \geq -\underline{a}, \quad c \geq 0.$$

Objects to solve for

- Value function $v(a, \epsilon)$
- Asset policy $a'(a, \epsilon)$
- Consumption policy $c(a, \epsilon) = (1 + r)a + w\epsilon - a'(a, \epsilon)$

With CRRA utility $u(c) = c^{1-\sigma}/(1-\sigma)$ and standard assumptions on π , the value function is increasing, concave in a , and the policy is monotone.

Budget constraint and asset structure

- a denotes a one-period risk-free claim that pays $(1 + r)a$ next period.
- **Borrowing constraint:** $a' \geq -\underline{a}$.
 - *Natural* bound: $\underline{a} = w\epsilon_{\min}/r$ (largest debt repayable from minimum income).
 - *Ad-hoc* bound: any $\underline{a} \leq$ natural bound.
- Without uncertainty, the household would set $\beta(1 + r) = 1$ and consumption-smooth perfectly. With uninsurable risk and a binding constraint, the **precautionary motive** drives extra saving.
- This precautionary saving raises aggregate capital relative to the complete-markets benchmark, and depresses r below $\rho = 1/\beta - 1$.

A representative firm operates a constant-returns technology

$$Y = F(K, L) = AK^\alpha L^{1-\alpha}.$$

Competition in factor markets gives

$$r = F_K(K, L) - \delta = \alpha A(K/L)^{\alpha-1} - \delta, \quad w = F_L(K, L) = (1 - \alpha)A(K/L)^\alpha.$$

- Aggregate labour supply $L = \sum_{\epsilon} \epsilon \pi^*(\epsilon)$ is exogenous.
- Aggregate capital K is determined endogenously by household saving.
- In equilibrium, K and r are jointly pinned down by household and firm decisions.

Stationary recursive competitive equilibrium (SRCE) — prices

A **stationary recursive competitive equilibrium** consists of:

1. Factor prices (r^*, w^*) derived from the firm's first-order conditions:

$$r^* = \alpha A(K^*/L)^{\alpha-1} - \delta, \quad w^* = (1 - \alpha)A(K^*/L)^\alpha.$$

2. Value function $v(a, \epsilon)$ and policies $(c(a, \epsilon), a'(a, \epsilon))$ that solve the household's problem given (r^*, w^*) .
3. A stationary cross-sectional distribution $\lambda^*(a, \epsilon)$.
4. Market clearing conditions (next slide).

SRCE — HH problem, market clearing, stationary distribution

Labour market clearing:

$$L = \sum_{\epsilon} \int \epsilon d\lambda^*(a, \epsilon).$$

Capital market clearing:

$$K^* = \sum_{\epsilon} \int a'(a, \epsilon) d\lambda^*(a, \epsilon).$$

Stationarity of the distribution:

$$\lambda^*(A', \epsilon') = \sum_{\epsilon} \pi(\epsilon, \epsilon') \int_{\{a: a'(a, \epsilon) \in A'\}} d\lambda^*(a, \epsilon).$$

That is, λ^* is invariant under the Markov operator induced by the policy $a'(\cdot)$ and the shock transition π .

Stationarity and the invariant measure

- Time-invariant prices require a time-invariant cross-sectional distribution.
- The pair $(a'(\cdot), \pi)$ defines a Markov **transition operator** Q on the state space $S = \mathbb{R}_+ \times \mathcal{E}$:

$$(Qf)(a, \epsilon) = \sum_{\epsilon'} \pi(\epsilon, \epsilon') f(a'(a, \epsilon), \epsilon').$$

- The invariant distribution λ^* satisfies $\lambda^* = Q^* \lambda^*$ (where Q^* is the adjoint).
- Existence and uniqueness of λ^* are non-trivial: they depend on **Feller** continuity, **mixing**, and **monotonicity** of Q — the next few slides.

Two markets

- Aiyagari's equilibrium reduces to clearing in **two markets**:
 - *Labour market*: trivially satisfied because labour supply is inelastic.
 - *Capital market*: aggregate household savings = aggregate firm demand for capital.
- All the action is in the capital market.
- Strategy: parametrise everything by r .
 - For each candidate r , solve the household problem to get $a'(a, \epsilon; r)$.
 - Compute the implied stationary distribution λ_r .
 - Compute the supply of capital $A(r) = \int a d\lambda_r$.
 - Compare to the demand $K(r)$ from the firm side; iterate.

Determination of r : demand for capital

The firm's first-order condition gives a downward-sloping demand for capital:

$$K(r) = L \left(\frac{r + \delta}{\alpha A} \right)^{\frac{1}{\alpha - 1}}.$$

- As $r \uparrow$, capital becomes more expensive and demand falls.
- $K(r)$ is continuous and strictly decreasing in r .
- $K(r) \rightarrow \infty$ as $r \rightarrow -\delta$ and $K(r) \rightarrow 0$ as $r \rightarrow \infty$.

This is the standard neoclassical capital demand schedule.

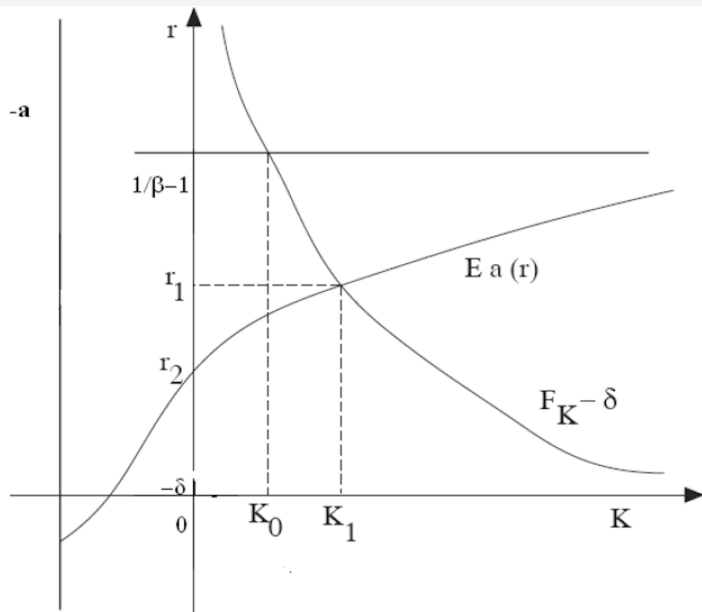
Determination of r : supply of capital

The household supply of capital is the cross-sectional average asset holding under the stationary distribution induced by r :

$$A(r) = \sum_{\epsilon} \int a d\lambda_r(a, \epsilon).$$

- $A(r)$ is upward-sloping in r (broadly).
- In the complete-markets benchmark $\beta(1+r) = 1$, $A(r)$ has a vertical asymptote at $r = 1/\beta - 1 = \rho$.
- With incomplete markets, precautionary saving makes $A(r)$ finite *below* ρ , so the equilibrium $r^* < \rho$.

Equilibrium picture



Probability measures and the Markov operator Q

- Let Λ denote the space of probability measures on the Borel sets of $S = \mathbb{R}_+ \times \mathcal{E}$.
- Define the Markov operator $Q : \Lambda \rightarrow \Lambda$ by

$$(Q\lambda)(B) = \int \mathbb{1}_{\{(a'(\epsilon), \epsilon') \in B\}} \pi(\epsilon, d\epsilon') d\lambda(a, \epsilon).$$

- Three properties of Q are essential to study the equilibrium:
 - **Feller property** $\rightarrow$ existence of an invariant measure.
 - **Mixing property** $\rightarrow$ uniqueness.
 - **Monotonicity** $\rightarrow$ convergence.
- These are standard conditions in the Stokey–Lucas–Prescott (1989) framework.

Feller property $\Rightarrow$ existence

Definition. Q has the *Feller property* if it maps continuous bounded functions to continuous bounded functions:

$$f \in C_b(S) \Rightarrow Qf \in C_b(S).$$

- Equivalently, the policy $a'(\cdot)$ is continuous in the state.
- Combined with compactness of the state space (or tightness of the orbit), Feller continuity guarantees the **existence of an invariant measure** via the Krylov–Bogolyubov theorem.
- In Aiyagari, the asset grid is bounded above by $w\epsilon_{\max}/r$ (since saving above this generates negative consumption probability), so S is effectively compact.

Mixing property $\Rightarrow$ uniqueness

Mixing condition. For every two states $s_1, s_2 \in S$, there exist $n_1, n_2 \in \mathbb{N}$ such that the n -step distributions $Q^{n_1} \delta_{s_1}$ and $Q^{n_2} \delta_{s_2}$ have overlapping support.

- Intuitively: starting from *any* two initial states, the chain visits a common region in finite time.
- This rules out absorbing sets that would split the state space and admit multiple invariant measures.
- **Counterexample:** a deterministic policy that pushes any household with $a > a^*$ to $a \uparrow$ and any household with $a < a^*$ to $a \downarrow$ creates two disjoint absorbing sets and *multiple* invariant distributions.
- In standard Aiyagari, idiosyncratic income shocks ensure mixing.

Monotonicity $\Rightarrow$ convergence

Monotonicity. If $\lambda \succeq \lambda'$ in the first-order stochastic dominance sense, then $Q\lambda \succeq Q\lambda'$.

- Combined with Feller and mixing, monotonicity implies that iterating Q from any initial distribution converges to the unique invariant measure λ^* .
- This is the foundation for *simulating* the model: start from an arbitrary distribution, simulate forward many periods, and read off λ^* .
- **Counterexample:** a non-monotone policy can produce stationary distributions that are sensitive to the initial condition; iteration may cycle rather than converge.
- Concavity of the value function and convexity of the budget set yield monotone policies in Aiyagari.

Computation of Stationary Equilibrium

Aggregate precautionary savings, applications

- Because precautionary saving raises $A(r)$, the equilibrium interest rate satisfies $r^* < \rho = 1/\beta - 1$.
- Aggregate capital is **higher** than in the complete-markets benchmark.
- Standard applications of Aiyagari-style models:
 - Wealth and income inequality (top 10%, Gini, ...)
 - Welfare effects of social insurance and tax/transfer policy
 - Effects of inflation on wealth distribution
 - Idiosyncratic-risk asset pricing
 - Quantitative theories of entrepreneurship
- Krueger–Mitman–Perri (2016), Kaplan–Moll–Violante (2018) HANK, ...

Computation of SRCE: three-step algorithm

Goal: solve for $(r^*, w^*, \lambda^*, v, a', c)$ jointly.

1. **Step 1.** Fix a candidate r . Solve the household dynamic programming problem to obtain v_r and policies (a'_r, c_r) .
2. **Step 2.** Given a'_r , compute the stationary cross-sectional distribution λ_r as the eigenvector of the Markov transition matrix.
3. **Step 3.** Evaluate the excess demand for capital

$$d(r) = K(r) - \int a d\lambda_r,$$

and update r via bisection (or Brent) until $|d(r)| < \text{tol}$.

This is the **nested fixed point** that has been the workhorse of HA computation since Aiyagari (1994).

Step 1: solve the household problem for fixed r

For given (r, w) , iterate the Bellman operator

$$(T_r v)(a, \epsilon) = \max_{a' \geq -\underline{a}} \left\{ u((1+r)a + w\epsilon - a') + \beta \sum_{\epsilon'} v(a', \epsilon') \pi(\epsilon, \epsilon') \right\}$$

on a grid for a .

- Standard tools: VFI with monotonicity exploitation, Howard policy iteration, endogenous-grid method (Carroll), Coleman–Reffett operator.
- Concavity of u and the convex budget set guarantee a unique optimum.
- Output: $a'_r(a, \epsilon)$, $c_r(a, \epsilon)$, $v_r(a, \epsilon)$.

Step 2: stationary distribution from the transition matrix

Discretise a on a grid; combined with ϵ , this gives a finite state space of size $N = N_a \cdot N_\epsilon$.

The policy and shock transition induce a Markov matrix $\mathbf{Q}_r \in \mathbb{R}^{N \times N}$ with

$$\mathbf{Q}_r[(a_i, \epsilon_j) \rightarrow (a_k, \epsilon_\ell)] = \pi(\epsilon_j, \epsilon_\ell) \cdot \Pr(a'_r(a_i, \epsilon_j) = a_k).$$

The stationary distribution solves

$$\lambda_r = \mathbf{Q}_r^\top \lambda_r, \quad \mathbf{1}^\top \lambda_r = 1.$$

Computation: dominant left eigenvector of $\mathbf{Q}_r$, or simply iterate $\lambda \leftarrow \mathbf{Q}_r^\top \lambda$ until convergence (the monotonicity result above guarantees this works).

Step 3: market clearing on r

With λ_r in hand, compute the household supply of capital

$$A(r) = \sum_{i,j} a_i \lambda_r(a_i, \epsilon_j),$$

and compare to firm demand:

$$d(r) = K(r) - A(r).$$

Update rule:

- If $d(r) > 0$: capital is scarce, raise r .
- If $d(r) < 0$: capital is abundant, lower r .
- Bisection or Brent's method converges in $O(\log(1/\text{tol}))$ outer iterations.

Each outer iteration requires solving Steps 1 and 2 from scratch — the dominant cost.

Implementation details

- **Asset grid:** dense near the borrowing constraint where the policy bends; sparse for high a . Often log-spaced.
- **Initial value function:** zero, or the closed-form complete-markets value, or the value from the previous outer iteration.
- **Multigrid:** solve on a coarse grid first, then interpolate onto a finer grid as a warm start.
- **Acceleration:** Howard's policy iteration steps, modified policy iteration, endogenous grid method (Carroll, 2006).
- **Convergence criterion:** $\|v^{(n+1)} - v^{(n)}\|_{\infty} < \varepsilon_v$ inside; $|d(r)| < \varepsilon_r$ outside.

Numerical challenges and the curse of dimensionality

- For Aiyagari, the per-iteration state space is $\mathbb{R}_+ \times \mathcal{E}$ — 2 continuous dimensions, manageable on a grid.
- Real-world quantitative HA models often have:
 - Multiple assets (liquid + illiquid in HANK; portfolio choice).
 - Multidimensional idiosyncratic states (income, unemployment, family structure).
 - *Aggregate* shocks $\rightarrow$ the cross-sectional distribution becomes part of the state (Krusell–Smith).
- With d continuous individual states and N_a grid points per dimension, total points are N_a^d — the **curse of dimensionality**.
- This is where machine learning, and in particular deep neural networks, become indispensable.

Why ML? Motivating Example

Why approximate the value function with a neural network?

- Grid-based methods scale exponentially: N_a^d points for d continuous dimensions.
- Even modest models (income $\times$ wealth $\times$ housing $\times$ aggregate) become intractable.
- **Neural networks:**
 - Universal approximators with smooth basis functions.
 - Number of parameters scales (essentially) linearly in input dimension.
 - Compatible with stochastic gradient training and GPU hardware.
- Strategy: parameterise V and g by neural networks Γ_{γ_v} and Γ_{γ_g} , then update (γ_v, γ_g) by training.
- We illustrate first on a simple deterministic optimal growth model, then scale up.

Optimal growth as a toy: NN approximations of V and g

The deterministic Bellman equation:

$$V(k) = \max_g \{ u(k, g(k)) + \beta V(f(k, g(k))) \}.$$

Parameterise:

$$V(k) \approx \Gamma_{\gamma_v}(k), \quad g(k) \approx \Gamma_{\gamma_g}(k).$$

Algorithm sketch:

- Sample states $k \sim d(k)$.
- Compute target values $\hat{V}(k)$ from the right-hand side using current networks.
- Update γ_v to minimise $(\Gamma_{\gamma_v}(k) - \hat{V}(k))^2$.
- Update γ_g by gradient ascent on the per-state objective.
- Iterate.

One-step value update ($T_{\text{sim}} = 1$)

Given current policy $g^{(n-1)}$ and value $\Gamma_{\gamma_v}^{(n-1)}$, define the bootstrap target

$$\hat{V}(k) = u(k, g^{(n-1)}(k)) + \beta \Gamma_{\gamma_v}^{(n-1)}(f(k, g^{(n-1)}(k))).$$

Then update by minimising the mean-squared Bellman residual:

$$\gamma_v^{(n)} \in \arg \min_{\gamma_v} \mathbb{E}_{k \sim d} \left[\left(\Gamma_{\gamma_v}(k) - \hat{V}(k) \right)^2 \right].$$

- Estimate the expectation by a Monte Carlo average over a batch $\{k_i\}_{i=1}^B$.
- One *epoch* = several gradient steps on the loss.
- This is essentially *fitted value iteration* (Bertsekas & Tsitsiklis 1996), with a NN function class.

One-step policy update

Given the just-updated value $\Gamma_{\gamma_v}^{(n)}$, improve the policy by maximising the per-state RHS over the parameters of the policy network:

$$\gamma_g^{(n)} \in \arg \max_{\gamma_g} \mathbb{E}_{k \sim d} \left[u(k, \Gamma_{\gamma_g}(k)) + \beta \Gamma_{\gamma_v}^{(n)}(f(k, \Gamma_{\gamma_g}(k))) \right].$$

- Gradient with respect to γ_g is computed automatically (backpropagation).
- One step of Adam (or any first-order optimiser) on a Monte Carlo batch.
- Alternating these updates is structurally identical to **actor–critic** reinforcement learning.

Choosing the sampling distribution $d(k)$

Question: on which states k should we evaluate the loss?

- *Uniform* on the bounded grid: simple but wasteful (most weight on irrelevant states).
- *Stationary distribution under the current policy*: focuses learning where the model actually spends time. Computed by simulation.
- *Importance sampling*: re-weight to keep the gradient unbiased while sampling from a convenient proposal $q(k)$:

$$\nabla_{\gamma} \mathbb{E}_d[L(k; \gamma)] = \mathbb{E}_q \left[\frac{d(k)}{q(k)} \nabla_{\gamma} L(k; \gamma) \right].$$

The choice of $d(k)$ has *first-order* implications for solution accuracy: a network trained on the wrong support generalises poorly.

Bias–variance tradeoff for $T_{\text{sim}} = 1$

Pros of one-step bootstrapping:

- Cheap: each evaluation costs a single network forward pass.
- Low variance.

Cons:

- High bias: the target $\hat{V}$ relies on the previous (imperfect) network.
- Errors compound across iterations if the network is far from the true V .

Multi-step rollouts ($T_{\text{sim}} > 1$):

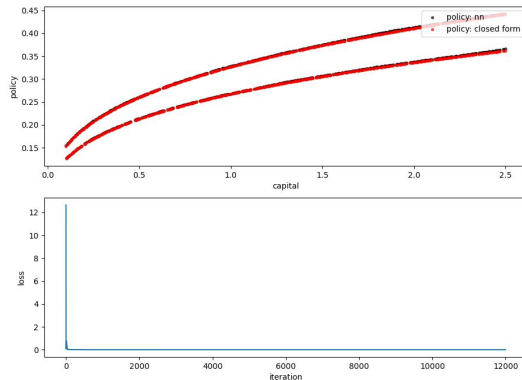
$$\hat{V}(k) = \sum_{t=0}^{T-1} \beta^t u(k_t, g(k_t)) + \beta^T \Gamma_{\gamma_v}(k_T).$$

- Reduces bias (less reliance on the bootstrapped tail).
- Increases variance and per-step cost.
- Choose T to trade off accuracy vs. wall time.

Improvement: stationary-distribution sampling

- Naive uniform sampling over a large bounded grid is wasteful: most points are never visited in equilibrium.
- Better: simulate the economy under the current policy until it converges to its (induced) stationary distribution, then sample k from that distribution.
- Benefits:
 - *Sample efficiency*: training data concentrates where the model lives.
 - *Faster convergence*: gradients informative.
 - *LLN*: large simulation $\Rightarrow$ low-variance estimates of expectations.
- Caveat: the sampling distribution *depends on the current policy*, which itself is being trained — creates a moving target. Standard fix: refresh the simulation every few epochs.

An illustration: optimal growth under stochastic productivity



- Stationary distribution of capital under the trained policy, vs. analytical benchmark.
- NN-trained policy matches the benchmark closely after a few thousand epochs of alternating updates.

What does RL/DL give us beyond grids?

- **Function approximation that scales with data, not with grid size.** A 4-layer MLP with a few thousand parameters can represent value functions in $\mathbb{R}^{20}$ that no Cartesian grid could store.
- **Stochastic gradient training.** Each gradient step uses a small batch; we can train indefinitely on streaming Monte Carlo data.
- **Hardware acceleration.** Forward and backward passes are GPU-friendly matrix multiplications.
- **End-to-end differentiability.** Bellman residuals, Euler residuals, and even simulation steps can be differentiated through.
- These advantages compound when we move from optimal growth $\rightarrow$ Aiyagari $\rightarrow$ Krusell–Smith $\rightarrow$ HANK.

Euler Equation + ML

Euler equation as an alternative optimality condition

For the optimal growth model with $u'(c) > 0$ and the budget constraint $c + k' = f(k)$, the consumer's first-order condition is the **Euler equation**:

$$u'(c(k)) = \beta u'(c(k')) f'(k').$$

- Equivalent to the Bellman equation (under standard regularity).
- Does not require representing V explicitly: we can train only the policy network.
- For the stochastic case with shock z :

$$u'(c(k, z)) = \beta \mathbb{E}[u'(c(k', z')) (1 + r(k', z')) \mid z].$$

From Euler residual to MSE loss

Define the **Euler residual** at state k as

$$\varepsilon(k; \gamma_g) = 1 - \frac{\beta \mathbb{E}[u'(c'(k; \gamma_g)) (1 + r') \mid k]}{u'(c(k; \gamma_g))}.$$

The optimal policy makes $\varepsilon(k) \equiv 0$. Cast as supervised learning:

$$\gamma_g^* \in \arg \min_{\gamma_g} \mathbb{E}_{k \sim d} [\varepsilon(k; \gamma_g)^2].$$

- Loss is *differentiable* with respect to γ_g via autograd.
- Conditional expectation approximated by Monte Carlo over future shocks.
- No explicit value function required.

Minimising the Euler-residual loss

Algorithm:

1. Initialise γ_g randomly (e.g., Xavier/Glorot).
2. Sample a batch of states $\{k_i\}_{i=1}^B$ from $d(k)$.
3. For each k_i , compute $c_i = c(k_i; \gamma_g)$ and $k'_i = f(k_i) - c_i$.
4. Sample shocks z' and compute the Monte Carlo expectation in ε .
5. Compute the squared residual loss and its gradient wrt γ_g .
6. Update γ_g using Adam.
7. Repeat until convergence ($\sum_i \varepsilon_i^2 < \text{tol}$).

Why this is attractive: *first-order necessary conditions* are precisely what we want the policy to satisfy. The neural network is just a flexible function class.

Code 1/5: ValueNet architecture (PyTorch)

```
import torch
import torch.nn as nn

class ValueNet(nn.Module):
    """Two-hidden-layer MLP V_gamma(k)."""
    def __init__(self, n_hidden=64):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(1, n_hidden),
            nn.ReLU(),
            nn.Linear(n_hidden, n_hidden),
            nn.ReLU(),
            nn.Linear(n_hidden, 1),
        )

    def forward(self, k):
        return self.net(k)
```

Code 2/5: value update step (Bellman MSE)

```
def value_update_step(value_net, policy_net, k_batch,
                      beta, opt_v):
    """One gradient step on the Bellman MSE loss."""
    opt_v.zero_grad()
    c_now = policy_net(k_batch)
    k_next = production(k_batch) - c_now
    v_next = value_net(k_next).detach()
    target = utility(c_now) + beta * v_next
    v_now = value_net(k_batch)
    loss = ((v_now - target) ** 2).mean()
    loss.backward()
    opt_v.step()
    return loss.item()
```

Note the `.detach()` on `v_next`: we treat the bootstrap target as a constant.

Code 3/5: policy improvement (gradient ascent)

```
def policy_step(value_net, policy_net, k_batch,
                beta, opt_g):
    """Maximise  $u(c) + \beta V(k')$  with respect to policy params."""
    opt_g.zero_grad()
    c = policy_net(k_batch)
    k_next = production(k_batch) - c
    objective = (utility(c) + beta * value_net(k_next)).mean()
    loss = -objective # gradient ascent*
    loss.backward()
    opt_g.step()
    return objective.item()
```

The budget constraint $c + k' = f(k)$ is encoded inside the function so the policy net only outputs c .

Code 4/5: training loop

```
def train(value_net, policy_net, n_epochs, batch_size,
          beta, lr_v=1e-3, lr_g=1e-3):
    opt_v = torch.optim.Adam(value_net.parameters(), lr=lr_v)
    opt_g = torch.optim.Adam(policy_net.parameters(), lr=lr_g)

    history = {"v_loss": [], "obj": []}
    for epoch in range(n_epochs):
        k_batch = sample_states(batch_size)           # from  $d(k)$ 
        v_loss = value_update_step(value_net, policy_net,
                                   k_batch, beta, opt_v)
        obj = policy_step(value_net, policy_net,
                           k_batch, beta, opt_g)
        history["v_loss"].append(v_loss)
        history["obj"].append(obj)
    return history
```

Code 5/5: visualisation

```
import matplotlib.pyplot as plt

def plot_results(value_net, policy_net):
    k = torch.linspace(0.1, 5.0, 100).reshape(-1, 1)
    with torch.no_grad():
        v = value_net(k).squeeze().numpy()
        c = policy_net(k).squeeze().numpy()

    fig, ax = plt.subplots(1, 2, figsize=(10, 4))
    ax[0].plot(k.numpy(), v); ax[0].set_title("V(k)")
    ax[1].plot(k.numpy(), c); ax[1].set_title("c(k)")
    plt.show()
```

A few thousand epochs of alternating value/policy updates produce policies that match the analytical benchmark to within numerical tolerance.

Actor–critic perspective: this is RL

- The two-network alternating update is exactly the structure of **actor–critic** reinforcement learning:

$$\text{Critic: } \Gamma_{\gamma_v} \leftarrow \arg \min \|\Gamma_{\gamma_v} - \hat{V}\|^2,$$

$$\text{Actor: } \Gamma_{\gamma_g} \leftarrow \arg \max \mathbb{E}[u + \beta V].$$

- Difference vs. classical RL: in macro the environment (the model) is *known*, so we can compute targets analytically rather than by Monte Carlo over an unknown $P(s'|s, a)$.
- This makes deep-learning macro *model-based RL with a known model* — often called “deep dynamic programming”.

Global Solution via DNN

Why (deep) RL helps

Three reasons NN-based methods outperform grid methods on HA models:

1. **Flexible function approximation.**

No structured Cartesian grid; networks handle high-dimensional inputs gracefully.

2. **Sampling-based training.**

RL methods were designed for unknown environments; here we re-use the data-generation tools (rollouts, replay, importance sampling) even though our model is known. This avoids constructing exhaustive grids or closed-form distribution evolution.

3. **Hardware acceleration.**

Forward/backward passes through neural networks are highly parallel matrix operations that GPUs and TPUs execute efficiently. Grid VFI is much harder to parallelise.

Global solution: full distribution as part of the state

- In Aiyagari we exploited *stationarity*: the distribution λ was a fixed object and dropped out of the household state.
- Once we add aggregate shocks, λ_t becomes part of the dynamic state. The household problem is

$$V(a, z, \lambda) = \max_{c, a'} \{u(c) + \beta \mathbb{E}[V(a', z', \lambda') \mid z]\}.$$

- The law of motion for λ is

$$\lambda' = G(\lambda, Z, Z'),$$

itself induced by the policy $a'(\cdot, \cdot, \lambda)$ and the shock kernel.

- λ is an infinite-dimensional object — this is the heart of the curse of dimensionality in HA models with aggregate shocks.

Household problem with an evolving distribution

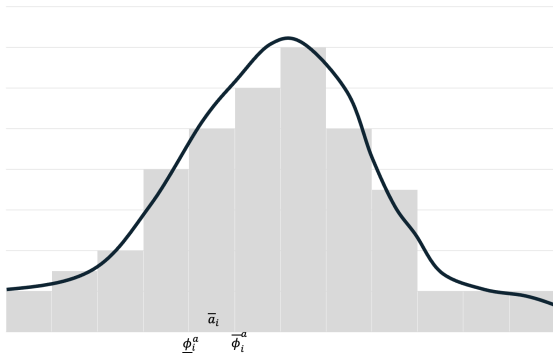
At the individual level, with state (a, z, λ) :

$$V(a, z, \lambda) = \max_{a'} \{ u((1 + r(\lambda))a + w(\lambda)z - a') + \beta \mathbb{E}[V(a', z', \lambda') | z] \}$$

with $\lambda' = G(\lambda, Z, Z')$.

- Two challenges:
 1. How to represent λ compactly enough that a NN can take it as input.
 2. How to evolve λ forward without simulating $\sim 10^6$ agents.
- Three families of solutions:
 - *Moment approximations* (Krusell–Smith): summarise λ by a few moments.
 - *Generalised moments* (DeepHAM): let the network learn the right summary statistics.
 - *Histogram transport* (DEQN): keep the full discretised distribution.

Approximating the distribution as a NN input



- One option: discretise λ on a fixed grid, feed the histogram heights to the NN.
- Another: pass a finite vector of moments (means, variances, percentiles).
- DeepHAM: *learn* basis functions $Q_1, \dots, Q_J$ and feed $\frac{1}{N} \sum_i Q_j(a_i)$ as moments.

DNN representation of V and the policy

Parameterise

$$V(S; \theta^V), \quad c(S; \theta^c), \quad S = (a, z, \lambda).$$

- Each network is a multilayer perceptron with parameters $\theta = (\text{weights, biases})$.
- Inputs: individual state (a, z) + summary of λ (moments or generalised moments).
- Outputs: scalar V , scalar consumption (or savings rate).
- Activations: ReLU, GELU, Mish — all popular for HA-style problems.
- Initialisation: Xavier/Glorot or Kaiming, depending on the activation.

Update value function via simulation + supervised learning

Given current policy $f_g^{(n)}$, generate N_V initial states $\{S_{i,0}\}_{i=1}^{N_V}$ and simulate T_V periods forward. Compute the empirical (truncated) discounted utility:

$$\hat{V}(S_{i,0}) = \sum_{t=0}^{T_V-1} \beta^t u(c_{i,t}).$$

Train the value network by supervised regression:

$$\theta^{V,(n+1)} \in \arg \min_{\theta^V} \sum_{i=1}^{N_V} (V(S_{i,0}; \theta^V) - \hat{V}(S_{i,0}))^2.$$

- This is sometimes called *Monte Carlo value estimation*.
- Larger T_V reduces bias but increases variance and cost.

Update policy with terminal value function

Given the just-updated value network $V(\cdot; \theta^{V,(n+1)})$, choose policy parameters to maximise the truncated objective with terminal value:

$$\theta^{c,(n+1)} \in \arg \max_{\theta^c} \mathbb{E} \left[\sum_{t=0}^{T_g-1} \beta^t u(c_{i,t}) + \beta^{T_g} V(S_{i,T_g}; \theta^{V,(n+1)}) \right]$$

subject to the budget constraint

$$c_{i,t} = (1 + r(\lambda_t)) a_{i,t} + w(\lambda_t) z_{i,t} - a'_{i,t}, \quad a'_{i,t} = c(S_{i,t}; \theta^c).$$

This is a long-horizon *differentiable rollout*: gradients propagate through the simulated path.

Transition function G as a matrix

On a discretised asset grid $\{a_1, \dots, a_{N_a}\}$, the policy $a'(\cdot)$ together with the shock kernel induces a Markov matrix

$$\hat{G}_{i,j} = \Pr(a' \in \phi_j^a \mid a \in \phi_i^a),$$

where ϕ_j^a are the discretisation cells.

- $\hat{G}$ has $O(N_a^2)$ entries; sparse if the policy moves between adjacent cells.
- Updating λ one period: $\lambda^{(t+1)} = \hat{G}^\top \lambda^{(t)}$.
- The same matrix is used in classical Aiyagari computation; here we re-use it inside the differentiable training loop.

Conditional and unconditional transition kernels

Unconditional:

$$\lambda'(z' \in Z', a' \in A') = \int_{Z \times A} \mathbb{K}\{a'(a, z, \lambda) \in A', z'(a, z, \lambda) \in Z'\} d\lambda(z, a).$$

Conditional (given current (a, z)):

$$\lambda'(z' \in Z', a' \in A' \mid z \in Z, a \in A) = \int_{Z \times A} \mathbb{K}\{\cdots\} d\lambda(z, a) / \lambda(Z \times A).$$

- Both kernels live in the Wasserstein space of probability measures on S .
- The induced *stochastic process on distributions* is the analytical object that DeepHAM and DEQN approximate by neural networks.

Discretised transition: i.i.d. shocks, monotone policy

When z is i.i.d. and the policy $a'(a, z)$ is monotone in a for each z , we can solve for transition cell boundaries explicitly:

$$\hat{G}_{i,j}^{\lambda, \eta_x} = \Pr(\phi_j^a \ni a_+ \leq \bar{\phi}_j^a \mid a \in \phi_i^a) = F_z(\bar{z}_{i,j}) - F_z(\underline{z}_{i,j}),$$

where $(\underline{z}_{i,j}, \bar{z}_{i,j})$ are the income shock thresholds that map ϕ_i^a to the boundaries of ϕ_j^a under the policy.

- Closed-form bin probabilities $\Rightarrow$ no Monte Carlo noise in the λ -update.
- Differentiability is delicate: the indicator functions inside the kernel are non-smooth, and need to be replaced by smooth approximations (e.g. sigmoids with sharp slopes) for gradient propagation.

Krusell–Smith (1998)

Krusell–Smith (1998): aggregate uncertainty

- Aiyagari ignores aggregate fluctuations. Krusell–Smith (*JPE* 1998) adds an **aggregate TFP shock** Z_t to the production technology.
- Setup:
 - Continuum of households with idiosyncratic employment $\epsilon \in \{0, 1\}$.
 - Aggregate state $Z_t \in \{Z_g, Z_b\}$, two-state Markov.
 - Joint (ϵ, Z) Markov chain (correlated unemployment and recessions).
 - Inelastic labour, single risk-free bond, borrowing constraint.
- The cross-sectional distribution Γ_t over (a, ϵ) now **moves stochastically over time**.

KS distributions, firms, prices

- Aggregate state: (Z, Γ) where Γ is the joint distribution over individual asset and employment states.
- Firm: $Y_t = Z_t F(K_t, \bar{L})$ with $K_t = \int a d\Gamma_t$ and $\bar{L}$ aggregate labour.
- Factor prices:

$$R_t = Z_t F_K(K_t, \bar{L}) - \delta, \quad w_t = Z_t F_L(K_t, \bar{L}).$$

- Both R_t and w_t depend on the aggregate state through (Z_t, K_t) , which depends on the entire *history* of the economy.

Household problem with the full distribution as state

The HH Bellman equation is now

$$V(a_i, y_i, Z, \Gamma) = \max_{c_i, a'_i} \{ u(c_i) + \beta \mathbb{E}[V(a'_i, y'_i, Z', \Gamma') \mid y_i, Z] \}$$

subject to

$$a'_i = w(Z, K) \bar{\ell} y_i + R(Z, K) a_i - c_i, \quad K = \int a d\Gamma,$$

and the law of motion $\Gamma' = G(\Gamma, Z, Z')$.

- **Curse of dimensionality:** Γ is infinite-dimensional.
- Without simplification, the household must *forecast the entire distribution* to make optimal decisions.

State reduction via finite moments

Krusell and Smith (1998) propose: replace Γ with a finite vector of moments

$$m = (m_1, m_2, \dots, m_J), \quad m_j = \int a^j d\Gamma.$$

- Most often: $J = 1$ — only the first moment $m_1 = K = \int a d\Gamma$.
- This dramatically reduces the household state from (a, y, Z, Γ) to (a, y, Z, K) .
- Approximation accuracy is excellent for many calibrations: agents care primarily about *average* capital because that determines r and w .
- “Approximate aggregation” (KS): higher moments rarely matter for the policy.

Postulated log-linear law of motion

Households need to forecast K_{t+1} given (Z_t, K_t) . Postulate

$$\log K_{t+1} = A(Z_t) + B(Z_t) \log K_t,$$

with state-dependent intercept $A(Z)$ and slope $B(Z)$.

- Two unknowns per aggregate state: $\{A(Z_g), B(Z_g), A(Z_b), B(Z_b)\}$.
- Households use this rule to compute $\mathbb{E}[V(a', y', Z', K') \mid y, Z]$.
- In equilibrium, the postulated rule must match the rule *generated* by the household policy under simulation.

KS algorithm: steps 1–2

Step 1. Specify the functional form for the law of motion. Initialise $\{A(Z), B(Z)\}$.

Step 2. Given the guessed law of motion, solve the household DP problem with state (a, y, Z, K) :

$$V(a, y, Z, K) = \max_{a'} \{ u(c) + \beta \mathbb{E}[V(a', y', Z', K'(Z)) \mid y, Z] \}$$

with $K'(Z) = \exp(A(Z) + B(Z) \log K)$.

- This is a 4-dimensional VFI problem (continuous in a and K , discrete in y and Z).
- Standard tools: tensor-product grids, multivariate interpolation, Howard improvement.¹

¹Classical HA computation methods are covered in detail in the video series at <https://space.bilibili.com/2142649036/lists/7180709>.

KS algorithm: steps 3–5

Step 3. Simulate the economy under the household policy: $T \approx 10,000$ periods, $N \approx 10,000$ agents. Record the time series $\{K_t, Z_t\}$.

Step 4. Run state-dependent OLS on the simulated $\{K_t, Z_t\}$:

$$\log K_{t+1} = \hat{A}(Z_t) + \hat{B}(Z_t) \log K_t + e_t.$$

Step 5. Update $\{A(Z), B(Z)\} \leftarrow \{\hat{A}(Z), \hat{B}(Z)\}$ and return to Step 2. Iterate until $|A_{\text{new}} - A_{\text{old}}| < \varepsilon$. **Convergence criterion:** in-sample R^2 of the regression close to 1.0 (often > 0.9999 in KS!).

Limitations of moment approximation

- Although remarkably accurate *on average*, the moment approximation has weaknesses:
 - *Specific functional form* for the law of motion is hard to choose ex ante.
 - *Higher-order properties* of the distribution (welfare, inequality measures) require more moments.
 - *Optimal-policy analysis* (designing fiscal/monetary intervention) needs derivatives of the value function with respect to the full distribution, not just moments.
- **The natural alternative:** let a neural network *learn* which features of Γ matter.
- This is exactly the DeepHAM / DEQN approach.

DeepHAM

DeepHAM (Han, Yang & E, 2025)

- DeepHAM solves the (finite- N , large- N) Krusell–Smith problem as a **Markov Nash equilibrium** among N agents.
- Two neural networks:
 - Value function $V(s; \theta^V)$ for individual states.
 - Policy $\mathcal{C}(s; \theta^C)$ for individual savings/consumption.
- Training alternates:
 1. **Value step**: simulate under current policy, compute Monte Carlo target, do supervised regression on θ^V .
 2. **Policy step**: solve agent i 's problem with all other agents fixed at the previous policy — *fictitious play*.
- This decomposition makes the high-dimensional joint problem tractable.

DeepHAM: algorithm outline

Initialisation ($k = 0$). Initial policy $\mathcal{C}^{(0)}$, networks $\theta^{V,(0)}, \theta^{C,(0)}$.

k -th iteration:

1. Prepare the stationary distribution $\mu(\mathcal{C}^{(k-1)})$ under the previous policy.
2. Update the value network by supervised learning:

$$\theta^{V,(k)} \in \arg \min_{\theta^V} \mathbb{E}_{\mu} \left[(V(s_i; \theta^V) - \hat{V}_i)^2 \right],$$

with target $\hat{V}_i = \sum_{\tau=0}^{T_v-1} \beta^{\tau} u(c_{\tau}^i)$.

3. Update the policy network by fictitious play:

$$\theta^{C,(k)} \in \arg \max_{\theta^C} \mathbb{E}_{\mu} \left[\sum_{t=0}^{T-1} \beta^t u(c_{i,t}) + \beta^T V(s_{i,T}; \theta^{V,(k)}) \right].$$

DeepHAM: value function learning

- Simulate the economy for T_v periods under the current policy $\mathcal{C}^{(k-1)}$.
- For each sampled initial state s_i , record the realised consumption path and compute the discounted utility target

$$\hat{V}_i = \sum_{\tau=0}^{T_v-1} \beta^\tau u(c_{i,\tau}).$$

- Train $V(\cdot; \theta^V)$ as a regression on $\{(s_i, \hat{V}_i)\}$.
- This is exactly fitted-value Monte Carlo: standard supervised deep learning, no special algorithms required.

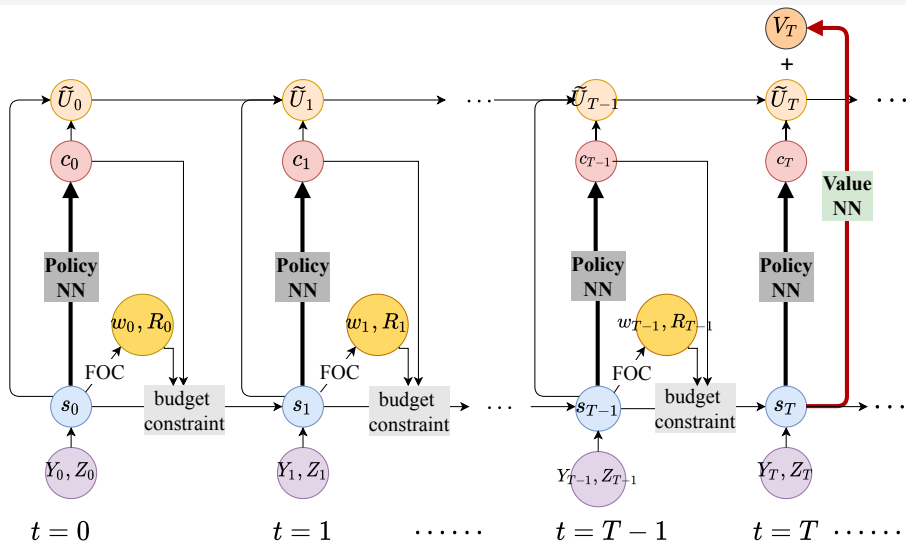
DeepHAM: policy iteration via fictitious play

- In round $\ell + 1$, all agents' policies $\mathcal{C}^{(k,\ell)}$ are known. Agent i improves *best-responding* to the previous policy of all others:

$$\mathcal{C}^{(k,\ell+1)} \in \arg \max_{\mathcal{C}} \mathbb{E}_{\mu} \left[\tilde{U}_{i,T} + \beta^T V^{(k)}(s_{i,T}) \right].$$

- Optimisation is over the parameters of the policy network and runs by gradient ascent on a Monte Carlo sample of paths.
- **Why fictitious play?**
 - Decouples the high-dimensional fixed-point problem.
 - Empirically robust: even in non-convex problems, alternating best response converges to a Nash equilibrium under regularity conditions.

DeepHAM: computational graph



- Inputs: individual state $s = (a_i, y_i, Z, \Gamma)$, with Γ summarised by generalised moments.

DL-based generalised moments

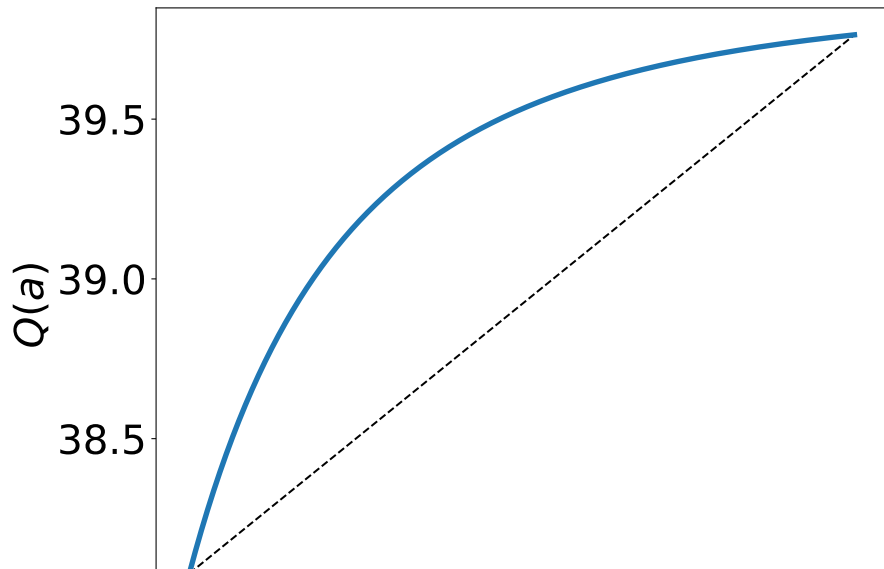
- Instead of a fixed list of moments $\{m_1, m_2, \dots\}$, DeepHAM lets the network *learn* symmetry-preserving sufficient statistics:

$$\bar{Q}_j(\Gamma) = \frac{1}{N} \sum_{i=1}^N Q_j(a_i),$$

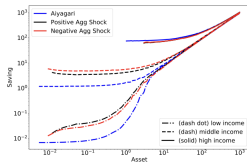
where $Q_1, \dots, Q_J$ are basis functions parameterised by additional NN layers.

- Special case $Q_j(a) = a^j$ recovers the classical moments.
- Trained jointly with V and $\mathcal{C}$, the learned moments adapt to the economic problem at hand.
- Symmetry across agents (the population is exchangeable) is enforced by averaging Q_j across the cross section.

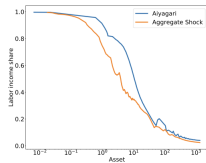
Interpretation of generalised moments



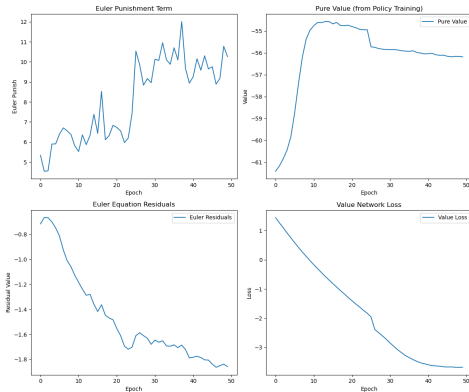
DeepHAM output figures



(a) Saving policy



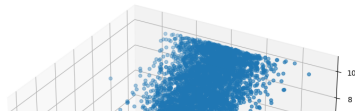
(b) Labor share



C Scatter Plot



A+ Scatter Plot



Deep Equilibrium Nets (DEQN)

Deep Equilibrium Nets (Azinovic, Gaegauf, Scheidegger 2022)

- DEQN solves the same KS economy with a **single** neural network, no value function.
- The single network represents the policy:

$$\sigma_{\theta}(a, e, Z, K) \in (0, 1) \Rightarrow a' = \underline{a} + \sigma_{\theta} \cdot (\text{cash on hand} - \underline{a}).$$

- The training objective is the **Euler equation residual**, not a Bellman residual.
- No value network, no fictitious play; just supervised learning on the Euler condition.
- Key innovation: a *cloud* of parallel economies provides a rich training distribution.

DEQN: policy network architecture

- Single multilayer perceptron $\sigma_{\theta} : \mathbb{R}^4 \rightarrow (0, 1)$.
- Inputs: (a, e, Z, K) where $K = \int a \, d\mu$ is aggregate capital.
- Architecture used in the original paper:
 - 3 hidden layers, 64 units each.
 - Mish activation function.
 - Sigmoid output for the savings rate.
- Sigmoid output enforces the budget constraint at the network level: the implied a' always lies within $[\underline{a}, \text{cash on hand}]$.
- Contrast with DeepHAM: DeepHAM uses both V_{NN} and $\mathcal{C}_{\text{NN}}$; DEQN uses only the policy network.

DEQN: Euler equation as the loss function

The household optimality condition is the Euler equation:

$$u'(c_t) = \beta \mathbb{E}_t[(1 + r_{t+1}) u'(c_{t+1})].$$

The **Euler residual**:

$$\varepsilon = 1 - \frac{\beta \mathbb{E}[(1 + r') u'(c')]}{u'(c)}.$$

The DEQN loss aggregates squared residuals across a sample of agents:

$$\mathcal{L}(\theta) = \frac{1}{M} \sum_{m=1}^M \varepsilon_m^2.$$

Why Euler equations? They are necessary and sufficient for optimality (under regularity); driving $\varepsilon \rightarrow 0$ trains the optimal policy without ever computing V .

DEQN: cloud simulation

Problem. To evaluate the Euler residual we need (K, Z) , but K depends on the cross-sectional distribution μ , which itself evolves endogenously under the trained policy.

Solution. Simulate N *independent copies* of the entire economy in parallel:

- Epoch 0: all N economies start at the steady-state distribution μ_{ss} from a sequence-Jacobian (SSJ) calibration.
- As training progresses, each economy evolves under its own TFP path $\{Z_n\}$, so the set $\{K_n\}$ spreads out across a realistic range.
- Result: an economically consistent and *diverse* batch on which the Euler loss is meaningful.

Compared to DeepHAM (one economy, N agents, T_v -step rollout), DEQN runs N economies, each with its own μ_n , evaluating the Euler equation directly each step.

DEQN: distribution transport

At each epoch, $\mu_{n,t}$ is a histogram over (a_j, e_k) . Steps to advance one period:

1. Compute savings $a'_{j,k}$ for every grid point under the current policy.
2. **Linear interpolation transport**: split mass $\mu(a_j, e_k)$ between the two adjacent grid points with weights $w_{\ell+1}, w_\ell$.
3. **Markov transition for employment**: distribute mass to (a_ℓ, e') weighted by the transition matrix $\Pi_{e \rightarrow e'}$.
4. Update $K_{n,t+1} = \sum_{j,k} a_j \cdot \mu_{n,t+1}(a_j, e_k)$.

The transport step runs inside `torch.no_grad()` so that gradients only propagate through the Euler residual loss, not through the simulation step itself.

DEQN: algorithm outline

Initialisation. Solve the steady state via SSJ to get K_{ss} , asset/employment grids, μ_{ss} , and the Markov transition Π .

For each training epoch $t \in \{1, \dots, T\}$:

1. Compute $K_n = \int a d\mu_n$ and read Z_n for each cloud copy n .
2. Sample M random (a, e) per copy.
3. Forward pass: evaluate $\sigma_\theta \rightarrow (a', c)$.
4. Compute Euler residuals ε^2 via expectations over (Z', e') .
5. Backward pass: Adam update on θ .
6. Transport: $\mu_n \rightarrow \mu'_n$ inside `no_grad()`; draw next Z_n .

Convergence. Loss decreases over ~ 2000 epochs; Euler errors below 10^{-3} indicate the policy solves the model to high accuracy.

DEQN vs. DeepHAM: comparison

	DeepHAM	DEQN
Networks	Value + policy (2)	Policy only (1)
Optimality	Bellman / fictitious play	Euler residual
Distribution rep.	Generalised moments	Full histogram
Training data	N agents, 1 economy	N economies, cloud sim
Forward sim	Yes, T_v steps	Yes, 1 step / epoch
Distribution evolution	Implicit via moments	Linear-interp transport
Scalability	Moments fixed; NN width grows	Histogram sized to grid
Differentiability	Through truncated rollout	Euler loss only

- Both avoid the curse of dimensionality of the underlying KS problem.
- DeepHAM has richer distributional info via learned moments; DEQN has a simpler architecture and direct Euler-based training.

Recap: the modern HA toolbox

- Aiyagari (1994): a non-trivial nested fixed point, classically solved by VFI + bisection on r .
- Krusell–Smith (1998): adds aggregate uncertainty, classically tamed by bounded-rationality moments.
- DeepHAM: replaces moments with *learned* sufficient statistics; alternates supervised value learning and fictitious-play policy improvement.
- DEQN: drops the value function entirely; trains a single policy network on Euler residuals over a cloud of simulated economies.
- Both deep-learning methods generalise straightforwardly to:
 - Multiple assets (HANK)
 - OLG with aggregate shocks (next part)
 - Mean-Field Games in continuous time (Part X)

OLG with Aggregate Uncertainty

From infinite-horizon to overlapping generations

- In Aiyagari/KS, agents are infinitely lived. Many quantitative questions require finite life cycles:
 - Social security and pension reform
 - Retirement savings and 401(k)
 - Lifecycle wealth accumulation
 - Education financing and student debt
- **OLG model** (Diamond, Auerbach–Kotlikoff): agents live J periods (or cohorts), are born, age, and die.
- State vector now includes the *age* of each cohort, plus its asset/income distribution.
- Standard OLG with aggregate shocks remains computationally hard — the per-cohort distribution is itself an infinite-dimensional state.

OLG with aggregate uncertainty: state vector

With J cohorts indexed by age $j = 1, \dots, J$:

- Cohort j has its own asset distribution Γ_j over (a_j, y_j) .
- Aggregate state: $S = (Z, \Gamma_1, \Gamma_2, \dots, \Gamma_J)$.
- Aggregate capital and labour:

$$K_t = \sum_{j=1}^J \int a d\Gamma_{j,t}, \quad L_t = \sum_{j=1}^J \int \bar{\ell}_j y d\Gamma_{j,t},$$

where $\bar{\ell}_j$ is the cohort-specific labour endowment (often hump-shaped).

- Lifecycle Bellman: $V_j(a_j, y_j, S) = \max \{u(c) + \beta \mathbb{E}[V_{j+1}(a', y', S')]\}$, with $V_{J+1} \equiv 0$ (or a terminal bequest function).

Why DL helps for OLG with aggregate shocks

- Without aggregate shocks: standard backward induction over cohorts works.
- **With** aggregate shocks: each cohort's continuation value depends on the future joint distribution of all cohorts $\Rightarrow$ exponential state-space blow-up.
- Deep neural networks let us:
 - Approximate $V_j(a, y, S)$ with a single MLP per cohort (or one MLP with cohort as an input).
 - Encode S via learned summary statistics (analogous to DeepHAM moments).
 - Train across cohorts in parallel on a GPU.
- Recent reference: Brumm, Kubler & Scheidegger (2023), "Economics-inspired neural networks with stabilizing homotopies".

Homotopy-stabilised training (Brumm–Kubler–Scheidegger 2023)

- OLG with aggregate uncertainty is hard to train: borrowing constraints, occasionally-binding bond limits, and discontinuities in the policy create unstable gradients.
- **Stabilising homotopy:** introduce constraints *gradually* during training.
 - Start with a relaxed (smooth) version of the model: no binding constraint, smooth penalties.
 - Slowly tighten the constraints toward their true value as training progresses.
 - By the time the constraints are at their true level, the network is already in a good basin.
- Empirically, homotopy is essential for getting OLG-with-aggregate-shocks training to converge.
- Implementation often combines JAX (for automatic differentiation through long simulation rollouts) with the Adam optimiser and a multistep homotopy schedule.

Lab12_OLG.ipynb preview

- Notebooks/Lab12_OLG.ipynb is a JAX-based implementation of the Brumm–Kubler–Scheidegger OLG model with stabilising homotopies.
- Highlights:
 - 7 cohorts spanning a 72-period life cycle, life-cycle labour endowment.
 - Aggregate TFP shocks following an AR(1).
 - Capital adjustment costs.
 - State-dependent depreciation (TFP-tied).
 - Two-stage homotopy: capital-only model first, then add bond.
- Use this notebook as an **optional follow-up** after Lab06: students who finish the KS lab early can explore OLG.
- In production research, the same toolkit handles multi-asset HANK and asset-pricing models.

Mean-Field Games

Mean-Field Games: motivation

- All the models we've seen so far are *finite or large-but-finite* populations of agents (Aiyagari has a continuum but discretises numerically).
- **Mean-Field Games** (MFG) provide a continuous-time, continuum-of-agents formulation.
- Why care?
 - Continuous time often *simplifies* the math: PDEs replace functional equations.
 - Powerful new toolbox for HA models with idiosyncratic and aggregate shocks.
 - Tight link to optimal transport, stochastic analysis, and modern PDE theory.
 - Recent breakthroughs in *deep learning for MFG* (Carmona–Laurière, Achdou–Lasry–Lions, Han et al.).
- This part is an introduction; you can think of MFG as the continuum limit of Aiyagari/KS.

The core idea: continuum + mean-field feedback

- Imagine a *continuum* of identical agents, each solving an individual control problem.
- Each agent's payoff depends on the *distribution* of all other agents (the “mean field”).
- The distribution is itself determined by the optimal behaviour of the agents.
- This circular dependence is the defining feature of MFG — and is exactly the same fixed point as in HA models, but stated in continuous time.
- Two mathematical objects describe the equilibrium:
 - A backward Hamilton–Jacobi–Bellman (HJB) equation for the individual value function.
 - A forward Kolmogorov forward / Fokker–Planck (KFE) equation for the distribution.

A brief history

- **Engineering origins** (mid-2000s):
Caines, Huang & Malhamé developed “Nash certainty equivalence” for large-population games.
- **Mathematical formalisation:**
Lasry & Lions (2006–2007) coined the term “Mean-Field Games” and developed the rigorous PDE theory.
- **Economic adoption:**
Achdou–Han–Lasry–Lions–Moll (2014, 2022 *REStud*) translated the framework into HA macroeconomics — continuous-time Aiyagari–Bewley–Huggett.
- **Deep-learning era** (2020s):
Carmona–Laurière, Han–Hu, Achdou–Pequito, . . . use deep neural networks to solve MFGs in high dimensions.

The two-equation MFG system

Hamilton–Jacobi–Bellman (backward, individual):

$$-\partial_t v(x, t) + \rho v(x, t) = H(x, \nabla v(x, t), m(\cdot, t)),$$

where H is the Hamiltonian of the individual control problem and $m(\cdot, t)$ is the population distribution at time t .

Kolmogorov forward / Fokker–Planck (forward, aggregate):

$$\partial_t m(x, t) = -\nabla \cdot (s^*(x, t, m) m(x, t)) + \frac{1}{2} \Delta (\sigma^2(x, t) m(x, t)),$$

where s^* is the optimal drift derived from the HJB.

- HJB and KFE are *coupled*: the optimal control depends on m , and m evolves under the optimal control.
- In equilibrium, both equations must hold simultaneously.

Why HJB is familiar, why KFE is new

- **HJB** is the continuous-time limit of the Bellman equation. Macroeconomists have seen it many times: Merton's portfolio choice, deterministic optimal growth, ...
- **KFE / Fokker–Planck** describes the time evolution of a probability density under stochastic dynamics.
 - Originated in physics: Fokker (1914), Planck (1917) — diffusion of particles.
 - Formalised by Kolmogorov (1931).
 - Standard in physics and engineering, only recently mainstream in macro.
- Together they describe the *joint dynamics of optimal individual behaviour and the cross-sectional distribution* — exactly what HA models need.

HJB review: deterministic optimal control

A generic optimal control problem:

$$v(x_0) = \max_{\alpha(\cdot)} \int_0^\infty e^{-\rho t} r(x_t, \alpha_t) dt \quad \text{s.t.} \quad \dot{x}_t = f(x_t, \alpha_t), \quad x_0 \text{ given.}$$

The corresponding HJB equation:

$$\rho v(x) = \max_{\alpha} \{r(x, \alpha) + v'(x) f(x, \alpha)\}.$$

Interpretation:

- LHS: “required return” on the value function.
- RHS: instantaneous payoff plus the marginal value of the state moving in the chosen direction.
- First-order condition over α : $r_{\alpha} + v'(x)f_{\alpha} = 0$.

From discrete-time Bellman to continuous-time HJB

Start with the discrete-time Bellman over a small interval Δ :

$$v(x) \approx \max_{\alpha} \{ r(x, \alpha) \Delta + e^{-\rho \Delta} v(x + f(x, \alpha) \Delta) \}.$$

Subtract $v(x)$ from both sides, divide by Δ , and let $\Delta \rightarrow 0$ (using $e^{-\rho \Delta} \approx 1 - \rho \Delta$ and $v(x + f \Delta) \approx v(x) + v'(x) f \Delta$):

$$0 = \max_{\alpha} \{ r(x, \alpha) - \rho v(x) + v'(x) f(x, \alpha) \}.$$

Rearrange:

$$\rho v(x) = \max_{\alpha} \{ r(x, \alpha) + v'(x) f(x, \alpha) \}.$$

This is exactly the HJB.

Stationary HJB and FOCs

For the optimal growth problem with state k , control c , return $u(c)$, and transition $\dot{k} = f(k) - \delta k - c$:

$$\rho v(k) = \max_c \{ u(c) + v'(k) (f(k) - \delta k - c) \}.$$

First-order condition:

$$u'(c) = v'(k) \implies c^*(k) = (u')^{-1}(v'(k)).$$

Substituting back:

$$\rho v(k) = u(c^*(k)) + v'(k) (f(k) - \delta k - c^*(k)).$$

A second-order ODE in v to be solved on $k \in [0, \bar{k}]$ with appropriate boundary conditions.

Poisson income shocks: state space

- In the continuous-time Aiyagari, individual income y jumps between two states $\{y_1, y_2\}$ at *Poisson rates* λ_1, λ_2 .
- State of an individual: (a, y_i) with $i \in \{1, 2\}$.
- Two value functions $v_1(a)$ and $v_2(a)$, one per income state.
- Asset evolves continuously: $\dot{a} = y_i + ra - c$.
- Income switches at Poisson rate; from state i , the agent moves to state $-i$ at rate λ_i .
- This setup is the natural continuous-time analogue of the discrete-time Aiyagari with Markov income.

Deriving the HJB with jump terms

Over a short interval Δ , with probability $\lambda_i \Delta$ the agent's income jumps from y_i to y_{-i} . The Bellman is

$$v_i(a) \approx \max_c \{ u(c) \Delta + e^{-\rho \Delta} [(1 - \lambda_i \Delta) v_i(a + \dot{a} \Delta) + \lambda_i \Delta v_{-i}(a)] \}.$$

Letting $\Delta \rightarrow 0$ and rearranging:

$$\rho v_i(a) = \max_c \{ u(c) + v'_i(a) (y_i + ra - c) + \lambda_i (v_{-i}(a) - v_i(a)) \}.$$

- Three terms: instantaneous utility, drift term (deterministic capital accumulation), jump term (Poisson income switch).
- This is the HJB with jumps; the system is $i \in \{1, 2\}$.

Economic intuition of the jump term

$$\lambda_i(v_{-i}(a) - v_i(a))$$

- This is the *option value* of switching income state.
- If $v_{-i}(a) > v_i(a)$ (the other income state would yield higher continuation value at the same wealth), the jump term is *positive* — the agent enjoys an expected gain from the future income switch.
- Conversely, if $v_{-i}(a) < v_i(a)$, the jump term is a cost.
- The Poisson rate λ_i controls the strength of this option value.
- Intuition: precautionary saving in continuous time appears partly through this jump term, in addition to the curvature of u along the deterministic drift.

Continuous-time Aiyagari: setup

Households. A continuum of households indexed by current asset $a \in [\underline{a}, \infty)$ and income $y_i \in \{y_1, y_2\}$ with Poisson switching rates λ_1, λ_2 .

Firm. Representative firm with $Y = K^\alpha L^{1-\alpha}$ and competitive factor markets:

$$r = \alpha(K/L)^{\alpha-1} - \delta, \quad w = (1 - \alpha)(K/L)^\alpha.$$

Aggregation. Aggregate capital and labour:

$$K = \sum_{i=1}^2 \int a g_i(a) da, \quad L = \sum_{i=1}^2 y_i \int g_i(a) da,$$

where $g_i(a)$ is the cross-sectional density in income state i .

The aggregation loop

1. Guess prices (r, w) (or equivalently aggregate K).
2. Solve the household HJB to get optimal consumption $c_i^*(a)$ and drift $s_i^*(a) = y_i + ra - c_i^*(a)$.
3. Solve the KFE for the stationary density $g_i^*(a)$.
4. Compute aggregate $K^* = \sum_i \int a g_i^*(a) da$.
5. Update (r, w) from the firm FOCs at K^* , and iterate until $K^* = K$.

This is structurally identical to the discrete-time Aiyagari nested fixed point, but every step is now an ODE/PDE rather than a discrete-time DP problem.

The Kolmogorov forward (Fokker–Planck) equation

For the time-dependent density $g_i(a, t)$:

$$\partial_t g_i(a, t) = -\partial_a(s_i(a, t) g_i(a, t)) - \lambda_i g_i(a, t) + \lambda_{-i} g_{-i}(a, t).$$

- First term: *drift* — mass moves at speed $s_i(a, t)$ along the a -axis.
- Second term: *Poisson outflow* from state i at rate λ_i .
- Third term: *Poisson inflow* from the other state at rate λ_{-i} .
- Boundary: at the borrowing limit $a = \underline{a}$, mass cannot leak below; we impose a reflecting boundary.
- Well-posed under standard conditions (Achdou et al. 2022).

Reading the KFE: economic intuition

- **Drift term:** the cross-section flows to the right when households are saving ($s_i > 0$) and to the left when they are dissaving.
- **Jump terms:** at every point in a , mass leaks from state i at rate λ_i and arrives from $-i$ at rate λ_{-i} . The two states “trade” agents.
- **Boundary at $\underline{a}$:** reflecting boundary — mass that hits the borrowing constraint cannot continue leftward; instead it accumulates at $\underline{a}$ for as long as $s_i(\underline{a}) \leq 0$.
- **Total mass conserved:** $\int g_1 + \int g_2 = 1$ at all times.
- Analogy: water flowing through two interconnected tanks with leaks at known rates.

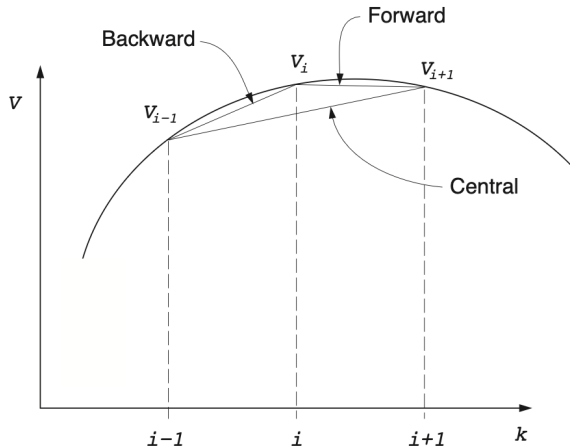
Stationary KFE

At the stationary equilibrium ($\partial_t = 0$):

$$0 = -\partial_a(s_i(a) g_i(a)) - \lambda_i g_i(a) + \lambda_{-i} g_{-i}(a), \quad i = 1, 2.$$

- Two coupled ODEs in g_1, g_2 , plus the normalisation $\int g_1 + \int g_2 = 1$.
- Reflecting boundary at $\underline{a}$, integrability at $a \rightarrow \infty$.
- Standard discretisation gives a sparse linear system $A^\top g = 0$ where A is the discretised HJB infinitesimal generator (next slide).

Discretisation and the upwind scheme



- Discretise a on a grid $\{a_1, \dots, a_N\}$. The HJB needs an approximation of $v'_i(a)$.
- **Upwind scheme:** pick the forward or backward difference depending on the *sign of the optimal drift*:

HJB and KFE: A vs. $A^\top$ duality

After upwind discretisation, the HJB takes the form

$$\rho \mathbf{v} = \mathbf{u}(\mathbf{c}) + A(\mathbf{v}) \mathbf{v},$$

where $A(\mathbf{v}) \in \mathbb{R}^{2N \times 2N}$ is the discretised infinitesimal generator (sparse, tridiagonal-plus-jumps).

The discretised stationary KFE is

$$\mathbf{0} = A(\mathbf{v})^\top \mathbf{g}.$$

- The *same* matrix A generates both equations: HJB on the right, KFE on the left (transpose).
- This duality is at the heart of the Achdou et al. approach: solving HJB gives you the KFE almost for free.

Putting it all together: outer fixed point on r

1. Guess r .
2. Solve HJB by iterating

$$\rho \mathbf{v} - A(\mathbf{v})\mathbf{v} = \mathbf{u}(\mathbf{c})$$

to find $\mathbf{v}^*, \mathbf{c}^*, A^*$.

3. Solve KFE: find $\mathbf{g}^*$ in the null space of $A^{*\top}$, normalised to $\mathbf{1}^\top \mathbf{g}^* = 1$.
4. Aggregate: $K^* = \sum_j a_j (g_{1,j}^* + g_{2,j}^*)$.
5. Update r from the firm FOC at K^* and iterate until convergence.

This is the Achdou–Lasry–Lions–Moll algorithm for continuous-time HA models. It is fast, accurate, and (with deep learning) extends to high-dimensional state spaces.

Transition dynamics: forward–backward system

For non-stationary problems (e.g. MIT shocks, business cycles), HJB and KFE become time-dependent and the system is *forward–backward*:

$$\begin{cases} -\partial_t v_i + \rho v_i = \max_c \{ \dots \}, & \text{(backward in time)} \\ \partial_t g_i = -\partial_a(s_i g_i) - \lambda_i g_i + \lambda_{-i} g_{-i}. & \text{(forward in time)} \end{cases}$$

- Boundary in time: $v_i(\cdot, T) =$ terminal condition (often the stationary v_i^*); $g_i(\cdot, 0) =$ initial condition.
- Numerical scheme: *shooting* or *Newton's method* on the residuals — iterate backward HJB given g , then forward KFE given v , until self-consistent.

Master equation: distribution as state

- The master equation is a single equation that encodes both HJB and KFE simultaneously, with the *distribution itself* as a state variable.
- Schematically:

$$\rho U(x, m) = H(x, \nabla_x U(x, m), m) + \int \nabla_m U(x, m)(y) \mathcal{L}^* m(y) dy,$$

where U is the value function defined on the product space (individual state, distribution) and $\mathcal{L}^*$ is the dual generator on the distribution.

- This is an infinite-dimensional PDE on the Wasserstein space of probability measures.
- Practical use: linearisation around the stationary equilibrium gives a tractable system for solving HA models with aggregate shocks (Bilal 2023; Alvarez–Lippi 2022).

Deep learning for MFG

- Classical numerical schemes (upwind, finite differences) suffer in high-dimensional state spaces.
- **Deep neural networks for MFG:**
 - Parameterise $v(x, m)$ and $m(x)$ by NNs.
 - Train by minimising the residual of the HJB and the KFE jointly.
 - Use Monte Carlo sampling instead of grids.
- Recent progress (Carmona–Laurière 2021, Laurière et al. 2022) shows that deep MFG solvers can handle 10–20 dimensional state spaces that are completely out of reach for grid methods.
- These methods unify the discrete-time RL approaches (DeepHAM, DEQN) with the continuous-time PDE perspective.

MFG $\leftrightarrow$ HA models: the link

- Discrete-time Aiyagari is the natural *discretisation* of continuous-time MFG with idiosyncratic Poisson income.
- Krusell–Smith with aggregate uncertainty corresponds to MFGs with a *common noise* (the aggregate shock).
- In both formulations:
 - Optimal individual behaviour depends on the cross-sectional distribution.
 - The cross-sectional distribution evolves under optimal behaviour.
 - Equilibrium is a fixed point of this circular dependence.
- MFG provides the cleanest mathematical formalism, while RL/DL provide the computational engine.
- The two approaches are now converging: deep MFG solvers borrow heavily from RL; modern deep RL borrows from continuous-time control theory.

Lab Preview and Wrap-up

Lab06 preview: Krusell–Smith via Deep Equilibrium Nets

- In the 1-hour lab you will train a Deep Equilibrium Net to solve the Krusell–Smith (1998) economy in PyTorch.
- Notebook: `Lab06_KS_HA.ipynb` (in this folder).
- By the end of the lab you will:
 - Initialise the model from a steady state computed via the sequence-Jacobian (SSJ) toolbox.
 - Build a PyTorch policy network for the savings rate.
 - Run the cloud-of-economies training loop.
 - Visualise the trained policy and the loss curves.
 - Diagnose convergence and accuracy.
- Hardware: a single GPU (Colab T4 / V100) is comfortable; CPU also works for short runs.

Lab06 step-by-step

Part 1. Initialise via sequence-Jacobian:

- Solve the steady state to get asset and employment grids, transition matrices, K_{ss}, μ_{ss} .

Part 2. PyTorch implementation:

- Define the policy network (3 hidden layers, Mish, sigmoid output).
- Encode the budget constraint inside the network.
- Wrap the model physics (transition dynamics) as a class.

Part 3. The training loop (cloud simulation):

- Simulate N parallel economies.
- Compute the Euler-residual loss.
- Backprop and Adam update.
- Transport the distribution forward (no grad).

Part 4. Analysis:

- Plot the loss history (log scale).

Optional follow-up: Lab12_OLG.ipynb

- Once you finish the KS lab, you may want to explore `Notebooks/Lab12_OLG.ipynb`.
- This notebook implements OLG with aggregate uncertainty using JAX and stabilising homotopies (Brumm–Kubler–Scheidegger 2023).
- Highlights:
 - Two-stage homotopy: first solve a capital-only model, then introduce bonds.
 - JAX `jit` compilation for fast training-simulation episodes.
 - Detailed pedagogical commentary inside the notebook.
- Useful preparation for research projects involving life-cycle, fiscal policy, or asset-pricing applications.

Recap of Lecture 6

- **Aiyagari–Bewley–Huggett**: incomplete-markets HA model and stationary equilibrium; existence/uniqueness via Feller, mixing, monotonicity.
- **Classical computation**: nested fixed-point on r ; VFI on the household, eigenvector for the distribution, bisection on the price.
- **Why ML?** Curse of dimensionality once we add aggregate shocks, multiple assets, or life-cycle structure.
- **Deep dynamic programming**: policy/value networks, Euler residual losses, actor–critic.
- **DeepHAM and DEQN**: two leading deep-learning solutions for KS.
- **OLG with aggregate uncertainty**: homotopy-stabilised JAX training.
- **Mean-Field Games**: continuous-time formulation, HJB+KFE, deep MFG solvers as the unifying frontier.
- **Next lecture**: Large Language Models — a different but equally fast-moving deep-learning frontier.